

UOS - Capstone

Week 6 - Diffusion

박상훈 ¹

2026학년도 2학기

¹Gwanak Lab Co., Ltd., Korea
E. sanghun.park@gwanaklab.co.kr

Introduction

Diffusion

Research Questions

Team Project

Wrap-up

Introduction

- 지난 시간: VAE를 학습시켜 $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 에서 뽑아 데이터를 생성하고, latent space를 그려 $p(\mathbf{z})$ 와 $q(\mathbf{z})$ 가 완전히 겹치지는 않는다는 것을 확인했습니다.
- 마지막 질문은 “prior를 꼭 $\mathcal{N}(\mathbf{0}, \mathbf{I})$ 로 정해야 할까?”였습니다.

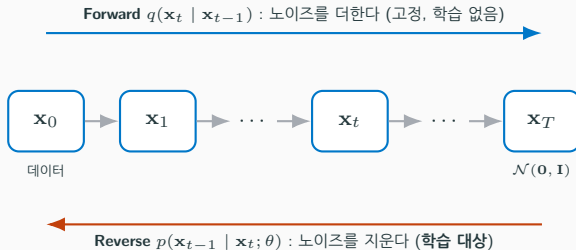
시간	내용	형태
~20분	왜 Diffusion인가	질문과 토론
~30분	Forward / Reverse Process	강의
~20분	구현 — 노이즈의 양, 시간의 주입	강의
~30분	MNIST 실습	실습 (RQ 1-4)
~10분	Tabular에의 적용 (TabDDPM)	강의 (RQ 5)
90분	조별 프로젝트	조별 작업

Diffusion

- VAE에서 $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 로 뽑았습니다. 분포에서 뽑는 것 자체가 이미 생성이므로, 우리는 가장 단순한 생성 모형 하나를 쓰고 있었던 셈입니다.
- 그 자리에 더 좋은 생성 모형을 끼우려면 샘플링이 쉬워야 하고 복잡한 모양도 표현할 수 있어야 합니다. GMM과 KDE는 차원이 커지면 무너졌습니다.
- 발상을 바꿔 봅시다. 데이터 $\rightarrow$ 노이즈는 그냥 더하면 되니 공짜입니다. 그 길을 아주 잘게 쪼개 두면, 한 스텝을 되돌리는 일은 쉬운 문제가 됩니다.

	VAE	Diffusion
생성 방식	$\mathbf{z}$ 하나에서 한 번에	노이즈에서 T 번에 걸쳐 조금씩
신경망 호출	1회 (빠름)	T 회 (느림)
한 번의 난이도	어렵다 $\Rightarrow$ 흐릿	쉽다 $\Rightarrow$ 선명

어려운 일을 한 번 하는 대신, 쉬운 일을 여러 번 한다.



- **Markov 성질:** 다음 상태가 바로 직전 상태에만 의존합니다. $q(\mathbf{x}_t | \mathbf{x}_{t-1}, \dots, \mathbf{x}_0) = q(\mathbf{x}_t | \mathbf{x}_{t-1})$.
- 덕분에 전체 경로가 한 스텝짜리 확률의 곱으로 쪼개집니다. $q(\mathbf{x}_{1:T} | \mathbf{x}_0) = \prod_t q(\mathbf{x}_t | \mathbf{x}_{t-1})$,
 $p(\mathbf{x}_{0:T}; \theta) = p(\mathbf{x}_T) \prod_t p(\mathbf{x}_{t-1} | \mathbf{x}_t; \theta)$.
- 즉 전체 변환을 한꺼번에 모델링할 필요가 없습니다. **한 스텝만** 모델링하면 됩니다.

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t \mid \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad 0 < \beta_1 < \dots < \beta_T < 1 \quad (1)$$

- 매 스텝 원래 값을 조금 줄이고($\sqrt{1 - \beta_t}$ 를 곱하고) 노이즈를 조금 더합니다. 줄이면서 더해야 분산이 발산하지 않고 1 근처로 유지됩니다.
- β_t 는 우리가 미리 정하는 상수입니다. Forward에는 신경망이 없습니다.
- $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$ 로 두면 가우시안끼리 합쳐지므로 **한 번에 점프**할 수 있습니다.

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t \mid \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}) \iff \mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (2)$$

$t = 700$ 을 보려고 700번 반복할 필요가 없습니다. $\bar{\alpha}_t$ 는 원본이 얼마나 남았는지입니다.

- 역방향 한 스텝도 가우시안으로 두고, 그 **평균만** 신경망으로 학습합니다. 평균을 직접 맞히는 대신 **더해진 노이즈 ϵ 을** 맞추게 두면 식이 간단해집니다.

$$p(\mathbf{x}_{t-1} \mid \mathbf{x}_t; \theta) = \mathcal{N}(\mathbf{x}_{t-1} \mid \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}), \quad \boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) \quad (3)$$

학습 목표는 결국 이것 하나입니다.

$$\mathcal{L} = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t) \right\|^2$$

생성 모델인데 손실 함수가 **그냥 MSE**입니다. 적대적 상대도, KL 항도 없습니다.

- 정해야 하는 것은 $\beta_1, \dots, \beta_T$ 뿐이지만, 실제로 중요한 것은 누적값 $\bar{\alpha}_t$ 입니다.

스케줄	정의	성질
linear	β_t 를 $10^{-4} \rightarrow 0.02$ 로 선형 증가	단순하지만 후반부가 이미 순수 노이즈라 낭비
cosine	$\bar{\alpha}_t$ 를 코사인 곡선으로 직접 설계	중간 구간이 완만. 쓸모 있는 스텝이 늘어남
quadratic	β_t 가 뒤로 갈수록 급증	초반 정보를 오래 보존

반드시 확인할 두 끝점: $\bar{\alpha}_T \approx 0$ (아니면 샘플링 시작점과 학습이 어긋남), $\bar{\alpha}_1 \approx 1$

- MNIST는 28×28 로 작습니다. $T = 1000$ 이 꼭 필요한지 직접 확인해 봅시다.

Diffusion : Time-step Embedding

- 신경망은 하나이고 모든 t 를 담당합니다. “지금 노이즈가 얼마나 섞여 있는지”를 반드시 알려줘야 합니다.
- t 를 그냥 스칼라로 넣으면 신경망이 잘 쓰지 못합니다. 여러 주기의 $\sin/\cos$ 로 펼쳐 벡터로 만들어 넣습니다.

```
def timestep_embedding(t, dim):          # t: (B,) 정수 텐서
    half = dim // 2
    freqs = torch.exp(-math.log(10000) * torch.arange(half, device=t.device) / half)
    args = t[:, None].float() * freqs[None]
    return torch.cat([torch.cos(args), torch.sin(args)], dim=-1)  # (B, dim)

temb = self.time_mlp(timestep_embedding(t, dim))  # Linear -> SiLU -> Linear
h = conv_block(x) + temb[:, :, None, None]      # 각 블록마다 더해 준다
```

가장 흔한 버그: embedding을 만들어 놓고 모델에 실제로 더하지 않는 것. 에러 없이 돌아가지만 샘플이 전혀 나오지 않습니다.

학습 (한 스텝)

1. $t \sim \mathcal{U}\{1, \dots, T\}, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2. $\mathbf{x}_t$ 를 한 번에 계산
3. $\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2$ 로 역전파

```
betas = make_schedule(T)
alphas = 1.0 - betas
alpha_bar = torch.cumprod(alphas, 0)  # 누적곱

t = torch.randint(0, T, (x0.size(0),))
eps = torch.randn_like(x0)
a_bar = alpha_bar[t].view(-1, 1, 1, 1) # 배치에 맞춰 reshape
```

샘플링 (생성)

1. $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2. $t = T, \dots, 1: \mathbf{x}_{t-1} = \mu_\theta(\mathbf{x}_t, t) + \sigma_t \mathbf{z}$
($t = 1$ 에서는 $\mathbf{z}$ 를 더하지 않음)

```
x_t = a_bar.sqrt() * x0 + (1 - a_bar).sqrt() * eps
loss = F.mse_loss(model(x_t, t), eps) # t를 꼭 넣는다
```

학습은 for문 없이 끝나고, 생성은 T 번의 for문이 필요합니다. 이 비대칭이 Diffusion의 장단점 전부를 만듭니다.

Research Questions

RQ 1. Visualize Noise Schedule (1)

Research Question 1

Diffusion의 절반(Forward)은 **신경망 없이** 지금 당장 돌려볼 수 있습니다.

- MNIST 이미지 하나를 골라 $t = 0, 100, 200, \dots, T$ 에서의 $\mathbf{x}_t$ 를 **한 줄로 나란히** 그리세요.
- 몇 번째 t 부터 **숫자를 알아볼 수 없게** 되나요?
- 같은 그림 아래에 $\bar{\alpha}_t$ 곡선을 그리고, 숫자가 사라진 지점의 $\bar{\alpha}_t$ 값을 확인하세요.
- $\mathbf{x}_T$ 의 평균과 분산을 재어 보세요. 정말 $\mathcal{N}(\mathbf{0}, \mathbf{I})$ 에 가깝나요?

간단한 답

- $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ 이므로, 알아볼 수 있는 정도는 **신호** $\sqrt{\bar{\alpha}_t}$ 와 **잡음** $\sqrt{1 - \bar{\alpha}_t}$ 의 **비율**로 정해집니다.
- 대체로 $\bar{\alpha}_t$ 가 0.1 아래로 내려가면 숫자를 알아보기 어려워집니다. linear 스케줄에 $T = 1000$ 이면 보통 $t = 400 \sim 600$ 구간입니다.
- $\mathbf{x}_T$ 의 평균이 0, 분산이 1에 가까우면 T 가 충분한 것입니다. 숫자의 흔적이 남아 있다면 T 를 늘리거나 β 를 키워야 합니다.
- Forward에는 학습이 없으므로, **모델을 만들기 전에** T 와 스케줄이 적절한지 여기서 먼저 확인할 수 있습니다.

$\mathbf{x}_T$ 에 숫자가 남아 있으면, 샘플링 시작점 $\mathcal{N}(\mathbf{0}, \mathbf{I})$ 와 학습이 어긋납니다.

RQ 2. Visualize Noise Schedule (2)

Research Question 2

같은 그림을 **linear**와 **cosine** 스케줄로 각각 그려 비교해 봅시다.

- 두 스케줄의 $\bar{\alpha}_t$ 곡선을 한 그림에 겹쳐 그리세요.
- RQ 1의 이미지 쿨을 두 스케줄에 대해 위아래로 붙여 놓으세요.
- 숫자가 사라지는 시점이 어떻게 달라지나요?
- 전체 T 스텝 중 “이미 다 망가져서 더 할 일이 없는” 구간은 각각 몇 퍼센트인가요?

간단한 답

- 스케줄이 바뀌는 것은 결국 $\bar{\alpha}_t$ 곡선의 모양 하나입니다.
- **linear**: 초반에 빠르게 떨어지고, 뒤쪽 20 ~ 30%는 이미 $\bar{\alpha}_t \approx 0$ 입니다. 그 구간은 다 망가진 상태에 노이즈만 더 섞는 구간이라 학습에서 배울 것이 거의 없고, 샘플링에서는 시간만 씁니다.
- **cosine**: 중간 구간을 완만하게 만들어 쓸 수 있는 스텝 수를 늘립니다. 해상도가 낮을수록 차이가 큼니다.
- 즉 스케줄을 고르는 일은 모델의 용량과 계산을 어느 구간에 배분할지 고르는 일입니다.
- MNIST처럼 작은 데이터에 $\text{linear} + T = 1000$ 은 과합니다. cosine을 쓰거나 T 를 줄이는 편이 낫습니다.

Research Question 3

학습된 모델로 **중간부터** 되돌려 봅시다.

- 원본 x_0 에 t^* 까지만 노이즈를 준 뒤, 거기서부터 reverse를 돌려 x_0 을 복원하세요.
- $t^* = 50, 200, 500, 800, T$ 로 각각 해보고 결과를 나란히 놓으세요.
- t^* 가 커질수록 복원 결과는 어떻게 변하나요? 원본과 같은 숫자가 나오나요?
- 어느 t^* 부터 다른 숫자가 나오기 시작하나요?

간단한 답

- t^* 가 작으면 원본과 거의 같은 이미지가 나옵니다. 같은 숫자, 같은 필체입니다.
- t^* 를 키우면 **같은 숫자지만 다른 필체**가 되고, 더 키우면 **아예 다른 숫자**가 나옵니다.
- 그 전환점은 $\bar{\alpha}_{t^*}$ 가 충분히 작아져 $\mathbf{x}_{t^*}$ 에 원본의 정보가 남지 않는 지점이며, RQ 1에서 “숫자가 사라진” 지점과 대체로 일치합니다.
- 같은 t^* 로 여러 번 돌리면 결과가 매번 다릅니다. reverse의 각 스텝마다 노이즈를 더하기 때문입니다.
- 즉 t^* 는 **원본을 얼마나 지키고 얼마나 새로 만들지**를 정하는 손잡이입니다.

합성데이터에서는 이것이 곧 **원본과의 닮은 정도**와 **privacy** 사이의 손잡이가 됩니다.

Research Question 4

이제 $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 에서 출발해 끝까지 걸어와 봅시다.

- 작은 U-Net(또는 MLP)으로 MNIST에 대해 학습시키세요. 구조보다 t 가 **모델에 들어가는지**가 훨씬 중요합니다.
- 샘플 64개를 격자로 그리세요.
- **학습 도중** 일정 간격으로 같은 시드의 샘플을 뽑아 두고, epoch에 따라 어떻게 변해 가는지 이어 보세요.
- 생성 과정 자체($\mathbf{x}_T \rightarrow \mathbf{x}_0$)도 몇 장 뽑아 **한 줄로** 그려 보세요.

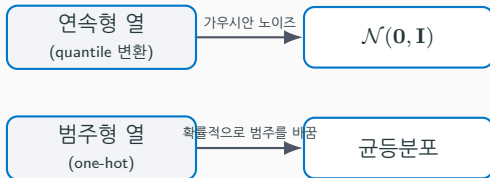
간단한 답

- 학습 초반에는 뭉개진 덩어리가, 이후에는 윤곽이, 마지막에 획이 선명해집니다. 같은 시드로 뽑아 두면 그 변화가 그대로 보입니다.
- 생성 과정을 한 줄로 늘어놓으면 큰 t 에서 전체 구조(밝은 영역의 위치와 크기)가 먼저 정해지고, 작은 t 에서 **획의 디테일**이 채워집니다.
- 그래서 스케줄이 중요합니다(RQ 2). 어느 구간에 스텝을 많이 두느냐가 **무엇을 잘 만드느냐**를 정합니다.
- 샘플이 전혀 나오지 않으면 순서대로 확인하세요. **(1)** t 가 모델에 실제로 들어가는가, **(2)** $\bar{\alpha}$ 를 cumprod로 구했는가, **(3)** 마지막 스텝에서 노이즈를 더하지 않았는가, **(4)** 정규화를 역변환했는가.

Research Question 5

MNIST에서 한 것을 정형데이터에 그대로 쓸 수 있을까요?

- 나이, 소득 같은 연속형 열은 어떻습니까? 큰 문제가 있을까요?
- 성별, 직업 같은 범주형 열은 어떻습니까? one-hot 벡터에 가우시안 노이즈를 더하면 (0.3, 0.4, 0.3) 같은 값이 됩니다. 이것을 “조금 망가진 범주”라고 할 수 있나요?
- Forward의 끝점이 무엇이어야 하는지 생각해 봅시다. 연속형의 끝점은 $\mathcal{N}(\mathbf{0}, \mathbf{I})$ 였습니다. 범주형의 끝점은 무엇이어야 할까요?



간단한 답

- 데이터 타입마다 “노이즈를 더한다”의 정의를 따로 두고, **하나의 모델로 함께** 학습합니다.
- **연속형**: quantile 변환 후 MNIST와 똑같은 가우시안 diffusion. **범주형**: 매 스텝 일정 확률로 다른 범주로 바뀌어 끝에는 모든 범주가 같은 확률 (multinomial diffusion).
- 두 손실을 **더해서** 하나의 신경망을 학습합니다. 신경망은 U-Net이 아니라 **MLP**입니다. 열에는 공간 구조가 없기 때문입니다.
- 차원이 낮아 T 가 작아도 되지만, **도메인 제약**(나이 ≥ 0 , 합계 일치)은 모델이 지켜주지 않습니다. 생성된 숫자를 **유효한 데이터로 되돌리는** 일이 절반입니다.

Team Project

목표: 우리 조의 합성데이터 생성 파이프라인을 설계하고, 오늘 안에 돌아가는 뼈대까지 만든다.

시간	할 일	산출물
0-15분	데이터와 목표 정하기	한 문장 목표
15-30분	설계 체크리스트 채우기	설계서 1장
30-70분	코드 뼈대 돌리기	1 epoch이라도 돌아가는 코드
70-85분	조별 발표 (조당 5분)	발표
85-90분	다음 주까지 할 일 정하기	역할별 to-do

오늘 완성할 필요는 없습니다. 막히는 지점을 정확히 찾아내는 것이 목표입니다.

Wrap-up

산출물

- t 에 따른 MNIST 노이즈 시각화 + $\bar{\alpha}_t$ 곡선 (RQ 1)
- linear와 cosine 스케줄 비교 그림 (RQ 2) □ t^* 별 복원 결과 비교 (RQ 3)
- 샘플 격자 + 생성 과정 한 줄 (RQ 4)

References

- J. Sohl-Dickstein et al., "Deep Unsupervised Learning using Nonequilibrium Thermodynamics," ICML, 2015.
- J. Ho, A. Jain, P. Abbeel, "Denoising Diffusion Probabilistic Models," NeurIPS, 2020.
- A. Nichol and P. Dhariwal, "Improved DDPM," ICML, 2021.
- E. Hoogeboom et al., "Argmax Flows and Multinomial Diffusion," NeurIPS, 2021.
- A. Kotelnikov et al., "TabDDPM," ICML, 2023. H. Zhang et al., "TabSyn," ICLR, 2024.

(cosine schedule)